E5c: G is not a small multiple of the first point on P-192 or P-224

2026-10-19. No ECDLP run. No security claim. An exact equality test of one stated derivation rule for the standard generator. Protocol `e5/PROTOCOL_E5c.md`, frozen at commit `8aab54912` before the example existed. Run report: `reports/run-20261019T2240Z/README.md`.

Question

Trapdoor program `nist_trapdoor_slot_program_20261011/PROTOCOL.md` §10, slot S4, experiment E4a lists the rule "`G = [h]·(first point)`". E5a (`E5.md`) excluded `P_first = ±[i]G` for `i ≤ 2^24`, which is `h ≡ ±i⁻¹ (mod n)`, and said plainly that small `h > 1` was not covered. E5c runs the direct check: is `G = ±[h]·P` for some `1 ≤ h ≤ 2^24`?

`P_first` is the point with the smallest valid abscissa: `x = 0` on P-192, `x = 3` on P-224 (recomputed by the example, same as the census). Both roots `y` are scanned. As an extension the first 16 valid abscissae of each curve are scanned, one root each (`±G` covers the other, since `[h](−P) = −[h]P`).

Result (measured)

curve	scans vs ±G	multiples compared	matches	most leading bits shared with `G.x`	controls (`±[12345]P_first`)
P-192	17	17 · 2^24 = 285 212 672	**0**	28 (of 192)	2 of 2 hit at `h = 12345`, correct sign, no other hit
P-224	17	17 · 2^24 = 285 212 672	**0**	28 (of 224)	2 of 2 hit at `h = 12345`, correct sign, no other hit

Checks before reading (all pass, `e5/e5c.json` → `checks`): `G` on the curve and `[n]G = 0`; all 16 base points on the curve; the incremental Montgomery path agrees with `Point::scalar_mul_vartime` on `x` and `y` of `[h]P_first` for `h ∈ {1, 2, 3, 1000, 12345, 2^16, 2^16+1, 2^20, 2^24}`, both roots, both curves (36 points, 0 mismatches).

The prediction ("no match") holds.

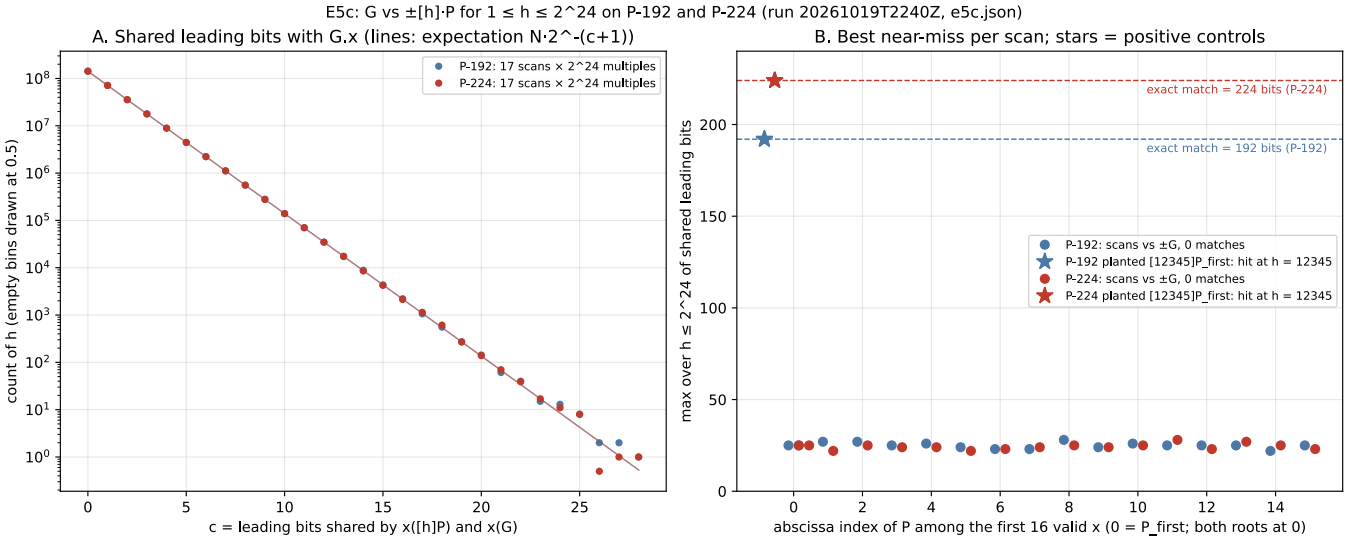


Figure 1: E5c figure

Figure. Run 20261019T2240Z, `e5/e5c.json`. A: for every scanned `[h]P`, the number of leading bits its abscissa shares with `G.x`, pooled over the 17 scans of each curve, against the protocol's expectation `N·2^{−(c+1)}`; the counts follow the line out to the largest values seen (about `log2 N ≈ 28`). B: the best near-miss per scan (about 22–28 bits) against the exact-match level ℓ; the stars are the planted controls, which reach ℓ. Source: `e5/plot_e5c.py`.

Panel A is the evidence that each scan visited `2^24` points that behave like distinct random points relative to `G.x` (a stuck or repeating loop would put mass far off the line); panel B shows the detector fires on a planted match.

What this closes, exactly

Closed, for P-192 and P-224: trapdoor slot S4, experiment E4a, rule "`G = [h]·(first point)`" for every `h` with `1 ≤ h ≤ 2^24` and either root of the first point (sign of `G` free: `G = ±[h]·P_first`). With E5a's inverse family this gives, for the first point, `G ≠ ±[h]·P_first` whenever `h ≤ 2^24` **or** `h⁻¹ mod n ≤ 2^24`.

Also excluded (extension): `G = ±[h]·P` for `h ≤ 2^24`, with `P` any of the first 16 valid abscissae (P-192: `x ∈ {0, 2, 3, 5, 6, 8, 12, 13, 15, 17, 18, 22, 26, 27, 30, 32}`; P-224: `x ∈ {3, 5, 8, 12, 13, 14, 17, 21, 23, 24, 25, 26, 27, 30, 31, 37}`).

Not closed: `h > 2^24` with $h^{-1} > 2^{24}$; base points other than these 16 abscissae (for example "first point" under another ordering or encoding); the SHA-1 rules of E4a (`x = SHA-1(seed ‖ i)`, the X9.62 `r` construction); E4b, the protocol audit. **S4 stays pending** on those items.

Provenance. P-192's `G` is already attributed to FreeLIP `zrandom` output (`nist_seed_provenance_20261016/README.md`); E5c's null is what that predicts. P-224's provenance remains unexplained; E5c removes one cheap mundane explanation and adds no other.

Scope and limits

An exact-match search over a stated finite range: it says nothing about `h` outside it, and makes no statement about the hardness of ECDLP on either curve. A null over `2^24` is a weak constraint against a rule with an arbitrary scalar; it is useful only because "small `h`" is a natural deterministic rule a designer could have used.

Graphs checked and left unchanged: `e5/e5.svg` (E5a/E5b; E5c adds no value to it).